Module 5: JDBC Programming Assignment

Connect a Database Using JDBC Driver —
Submitting Queries and Getting Results from the Database

Mausam Kar
Registration No: **24BAI10284**

April 11, 2026

Objective

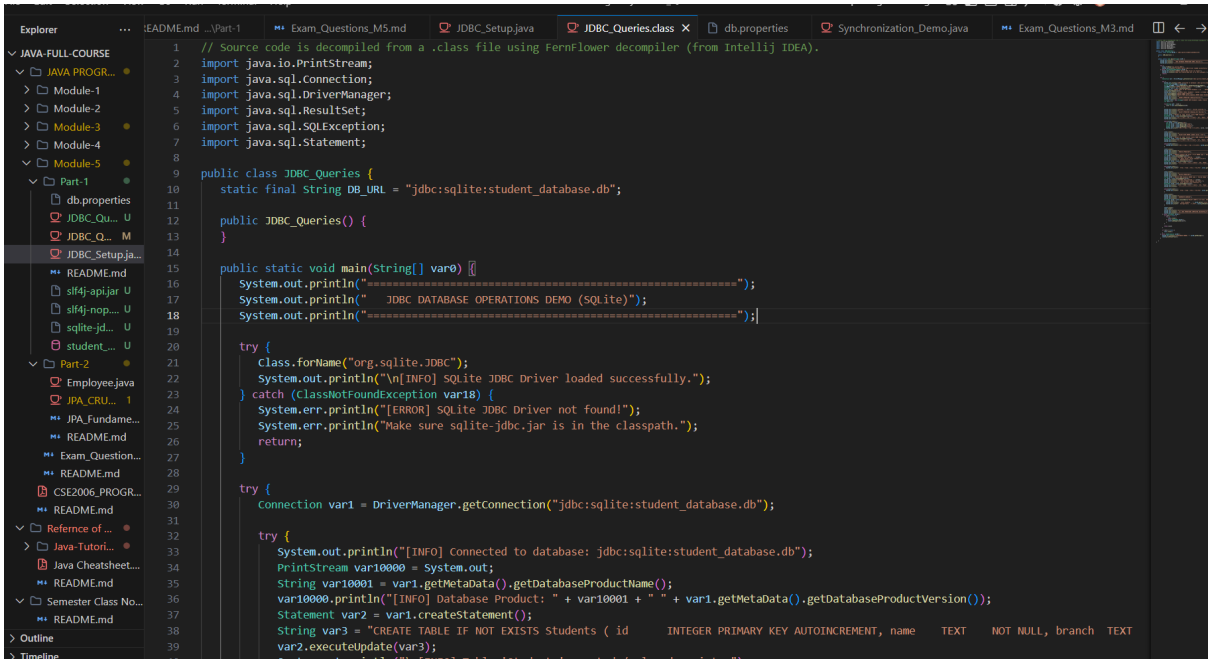
Write a Java program to connect a database by using a JDBC driver — submitting queries and getting results from the database. The program demonstrates all CRUD (Create, Read, Update, Delete) operations along with aggregate queries using JDBC with an SQLite embedded database.

Technologies Used:

- Java (JDK 21+)
- JDBC API (`java.sql.*`)
- SQLite Database (embedded, via `sqlite-jdbc` driver)

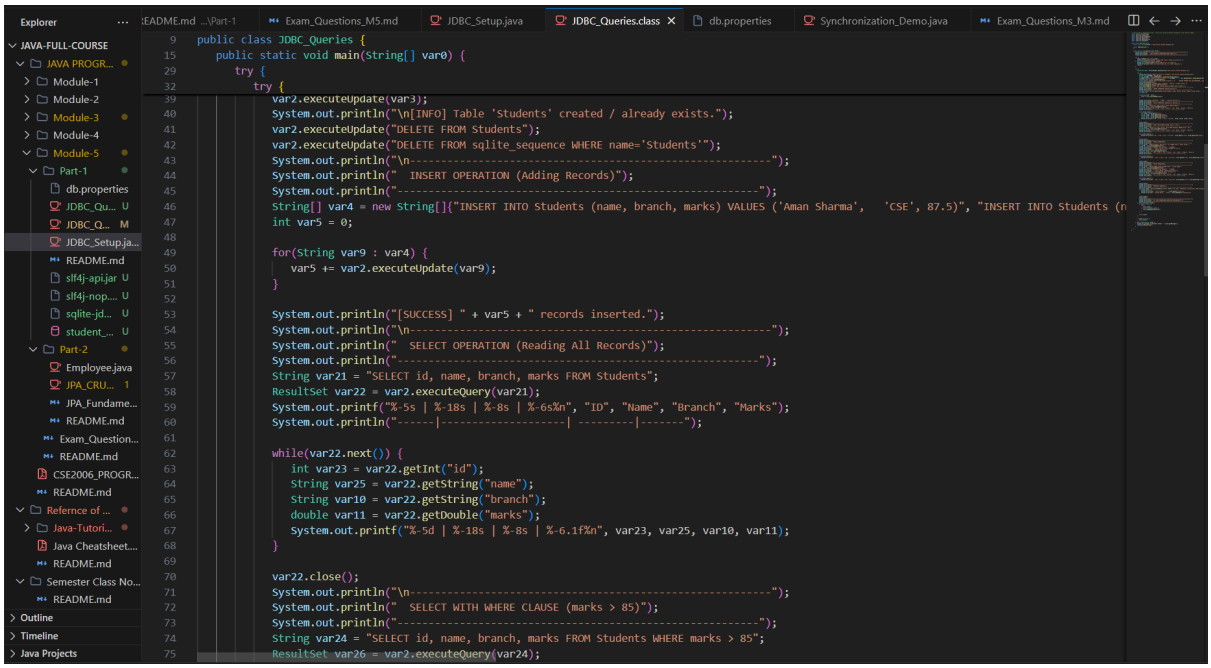
1 Program Code

The complete Java source code for the JDBC database operations program is shown below across multiple screenshots.



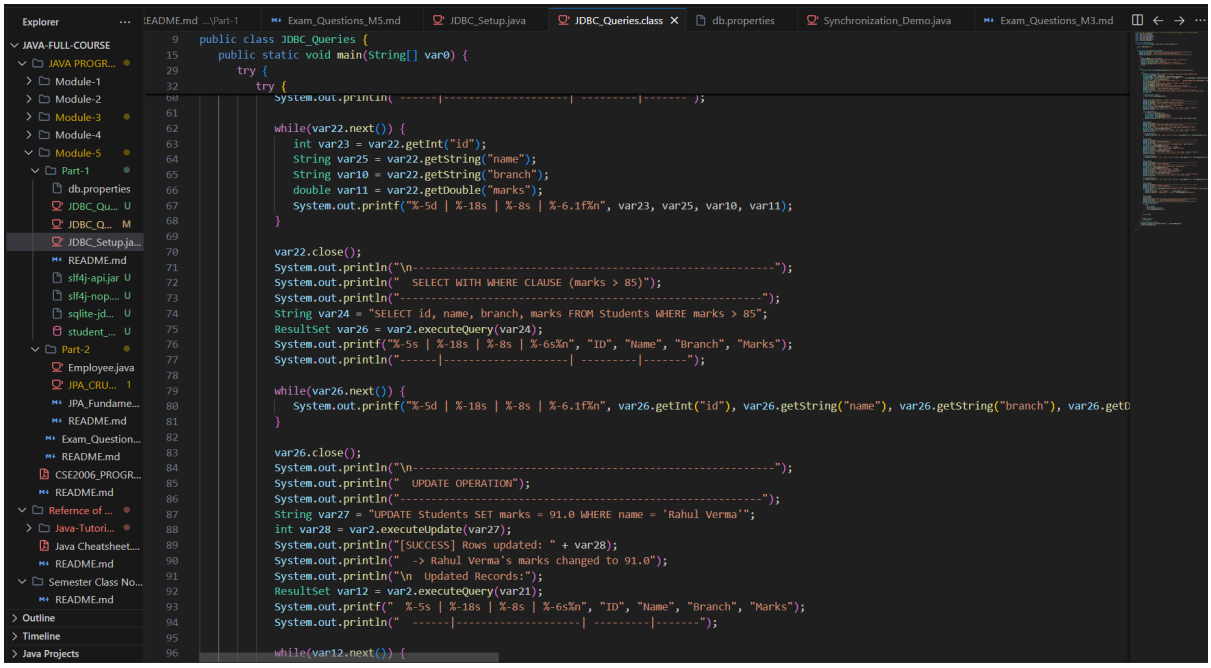
```java
// Source code is decompiled from a .class file using FernFlower decompiler (from Intellij IDEA).
import java.io.PrintStream;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

public class JDBC_Queries {
   static final String DB_URL = "jdbc:sqlite:student_database.db";

   public JDBC_Queries() {
   }

   public static void main(String[] var0) {
      System.out.println("=====================================================");
      System.out.println("   JDBC DATABASE OPERATIONS DEMO (SQLite)");
      System.out.println("=====================================================");

      try {
         Class.forName("org.sqlite.JDBC");
         System.out.println("\n[INFO] SQLite JDBC Driver loaded successfully.");
      } catch (ClassNotFoundException var18) {
         System.err.println("[ERROR] SQLite JDBC Driver not found!");
         System.err.println("Make sure sqlite-jdbc.jar is in the classpath.");
         return;
      }

      try {
         Connection var1 = DriverManager.getConnection("jdbc:sqlite:student_database.db");

         try {
            System.out.println("[INFO] Connected to database: jdbc:sqlite:student_database.db");
            PrintStream var10000 = System.out;
            String var10001 = var1.getMetaData().getDatabaseProductName();
            var10000.println("[INFO] Database Product: " + var10001 + " " + var1.getMetaData().getDatabaseProductVersion());
            Statement var2 = var1.createStatement();
            String var3 = "CREATE TABLE IF NOT EXISTS Students ( id      INTEGER PRIMARY KEY AUTOINCREMENT, name    TEXT    NOT NULL, branch  TEXT
            var2.executeUpdate(var3);
```

Figure 1: Code Part 1 – Imports, class declaration, driver loading, and connection setup



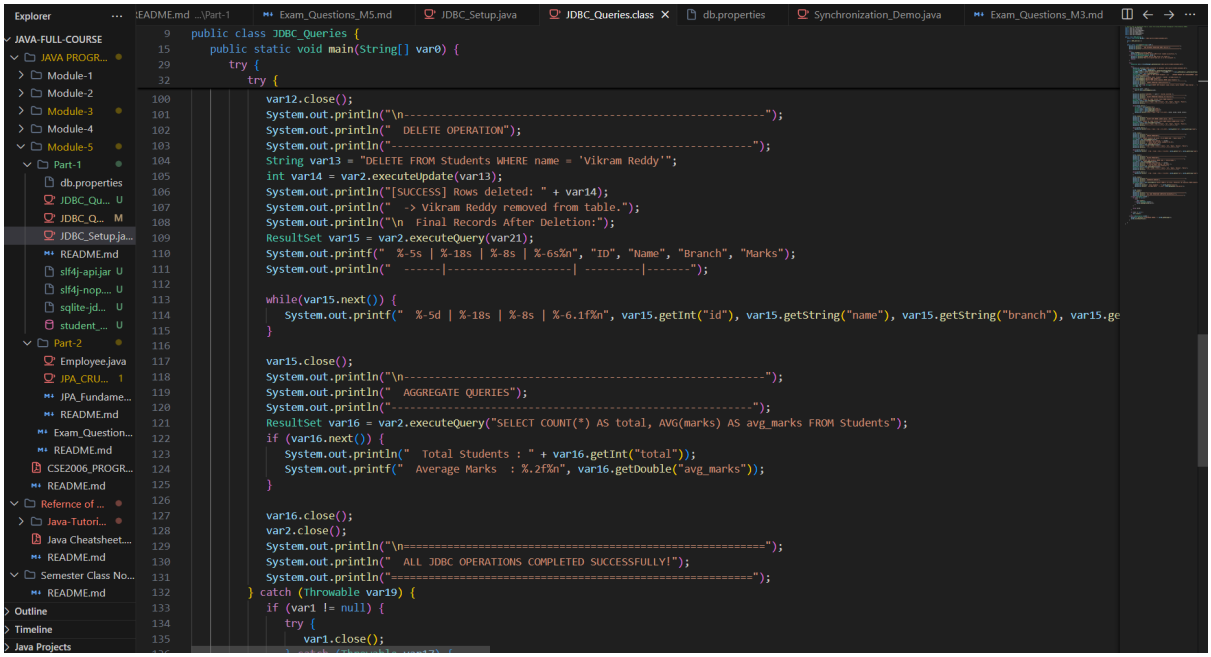
```java
public class JDBC_Queries {
   public static void main(String[] var0) {
      try {
         try {
            var2.executeUpdate(var3);
            System.out.println("\n[INFO] Table 'Students' created / already exists.");
            var2.executeUpdate("DELETE FROM Students");
            var2.executeUpdate("DELETE FROM sqlite_sequence WHERE name='Students'");
            System.out.println("\n-----------------------------------------------------");
            System.out.println("  INSERT OPERATION (Adding Records)");
            System.out.println("-----------------------------------------------------");
            String[] var4 = new String[]{"INSERT INTO Students (name, branch, marks) VALUES ('Aman Sharma',   'CSE', 87.5)", "INSERT INTO Students (n
            int var5 = 0;

            for(String var9 : var4) {
               var5 += var2.executeUpdate(var9);
            }

            System.out.println("[SUCCESS] " + var5 + " records inserted.");
            System.out.println("\n-----------------------------------------------------");
            System.out.println("  SELECT OPERATION (Reading All Records)");
            System.out.println("-----------------------------------------------------");
            String var21 = "SELECT id, name, branch, marks FROM Students";
            ResultSet var22 = var2.executeQuery(var21);
            System.out.printf("%-5s | %-18s | %-8s | %-6s%n", "ID", "Name", "Branch", "Marks");
            System.out.println("------|-------------------| --------|------");

            while(var22.next()) {
               int var23 = var22.getInt("id");
               String var25 = var22.getString("name");
               String var10 = var22.getString("branch");
               double var11 = var22.getDouble("marks");
               System.out.printf("%-5d | %-18s | %-8s | %-6.1f%n", var23, var25, var10, var11);
            }

            var22.close();
            System.out.println("\n-----------------------------------------------------");
            System.out.println("  SELECT WITH WHERE CLAUSE (marks > 85)");
            System.out.println("-----------------------------------------------------");
            String var24 = "SELECT id, name, branch, marks FROM Students WHERE marks > 85";
            ResultSet var26 = var2.executeQuery(var24);
```

Figure 2: Code Part 2 – CREATE TABLE and INSERT operations



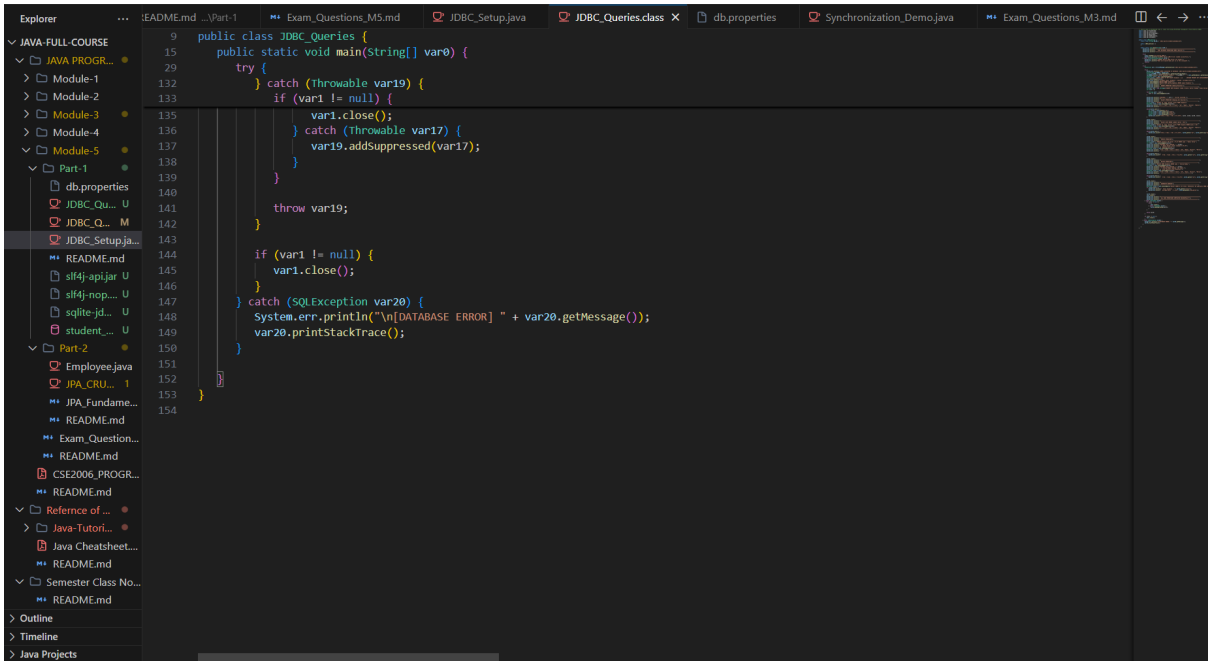
```java
public class JDBC_Queries {
    public static void main(String[] var0) {
        try {
            try {
                System.out.println("------|-------------------| ---------|------- );

                while(var22.next()) {
                    int var23 = var22.getInt("id");
                    String var25 = var22.getString("name");
                    String var10 = var22.getString("branch");
                    double var11 = var22.getDouble("marks");
                    System.out.printf("%-5d | %-18s | %-8s | %-6.1f%n", var23, var25, var10, var11);
                }

                var22.close();
                System.out.println("\n-------------------------------------------------------");
                System.out.println("  SELECT WITH WHERE CLAUSE (marks > 85)");
                System.out.println("-------------------------------------------------------");
                String var24 = "SELECT id, name, branch, marks FROM Students WHERE marks > 85";
                ResultSet var26 = var2.executeQuery(var24);
                System.out.printf("%-5s | %-18s | %-8s | %-6s%n", "ID", "Name", "Branch", "Marks");
                System.out.println("------|-------------------| ---------|-------");

                while(var26.next()) {
                    System.out.printf("%-5d | %-18s | %-8s | %-6.1f%n", var26.getInt("id"), var26.getString("name"), var26.getString("branch"), var26.getD
                }

                var26.close();
                System.out.println("\n-------------------------------------------------------");
                System.out.println("  UPDATE OPERATION");
                System.out.println("-------------------------------------------------------");
                String var27 = "UPDATE Students SET marks = 91.0 WHERE name = 'Rahul Verma'";
                int var28 = var2.executeUpdate(var27);
                System.out.println("[SUCCESS] Rows updated: " + var28);
                System.out.println("  -> Rahul Verma's marks changed to 91.0");
                System.out.println("\n  Updated Records:");
                ResultSet var12 = var2.executeQuery(var21);
                System.out.printf("  %-5s | %-18s | %-8s | %-6s%n", "ID", "Name", "Branch", "Marks");
                System.out.println("  ------|-------------------| ---------|-------");

                while(var12.next()) {
```

Figure 3: Code Part 3 – SELECT query with ResultSet traversal



```java
public class JDBC_Queries {
    public static void main(String[] var0) {
        try {
            try {
                var12.close();
                System.out.println("\n-------------------------------------------------------");
                System.out.println("  DELETE OPERATION");
                System.out.println("-------------------------------------------------------");
                String var13 = "DELETE FROM Students WHERE name = 'Vikram Reddy'";
                int var14 = var2.executeUpdate(var13);
                System.out.println("[SUCCESS] Rows deleted: " + var14);
                System.out.println("  -> Vikram Reddy removed from table.");
                System.out.println("\n  Final Records After Deletion:");
                ResultSet var15 = var2.executeQuery(var21);
                System.out.printf("  %-5s | %-18s | %-8s | %-6s%n", "ID", "Name", "Branch", "Marks");
                System.out.println("  ------|-------------------| ---------|-------");

                while(var15.next()) {
                    System.out.printf("  %-5d | %-18s | %-8s | %-6.1f%n", var15.getInt("id"), var15.getString("name"), var15.getString("branch"), var15.ge
                }

                var15.close();
                System.out.println("\n-------------------------------------------------------");
                System.out.println("  AGGREGATE QUERIES");
                System.out.println("-------------------------------------------------------");
                ResultSet var16 = var2.executeQuery("SELECT COUNT(*) AS total, AVG(marks) AS avg_marks FROM Students");
                if (var16.next()) {
                    System.out.println("  Total Students : " + var16.getInt("total"));
                    System.out.printf("  Average Marks  : %.2f%n", var16.getDouble("avg_marks"));
                }

                var16.close();
                var2.close();
                System.out.println("\n=======================================================");
                System.out.println("  ALL JDBC OPERATIONS COMPLETED SUCCESSFULLY!");
                System.out.println("=======================================================");
            } catch (Throwable var19) {
                if (var1 != null) {
                    try {
                        var1.close();
```

Figure 4: Code Part 4 – UPDATE and DELETE operations with result display



```java
public class JDBC_Queries {
    public static void main(String[] var0) {
        try {
            } catch (Throwable var19) {
                if (var1 != null) {
                    var1.close();
                } catch (Throwable var17) {
                    var19.addSuppressed(var17);
                }
            }

            throw var19;
        }

        if (var1 != null) {
            var1.close();
        }
    } catch (SQLException var20) {
        System.err.println("\n[DATABASE ERROR] " + var20.getMessage());
        var20.printStackTrace();
    }

}
}
```

Figure 5: Code Part 5 – Aggregate queries (COUNT, AVG) and closing

2 Program Output

The program was compiled and executed successfully. The output screenshots are shown below.

Commands used:

```
javac -cp ".;sqlite-jdbc.jar" JDBC_Queries.java
java -cp ".;sqlite-jdbc.jar;slf4j-api.jar;slf4j-nop.jar" JDBC_Queries
```

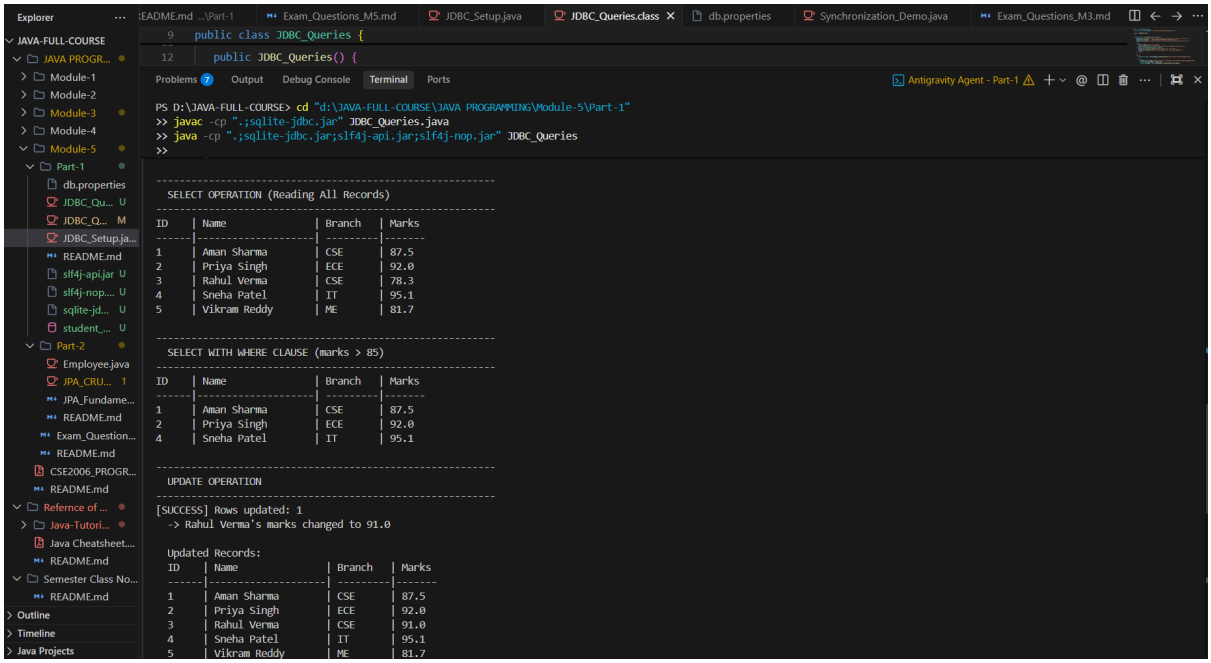


Figure 6: Output Part 1 – Connection, INSERT, SELECT all records, and filtered SELECT

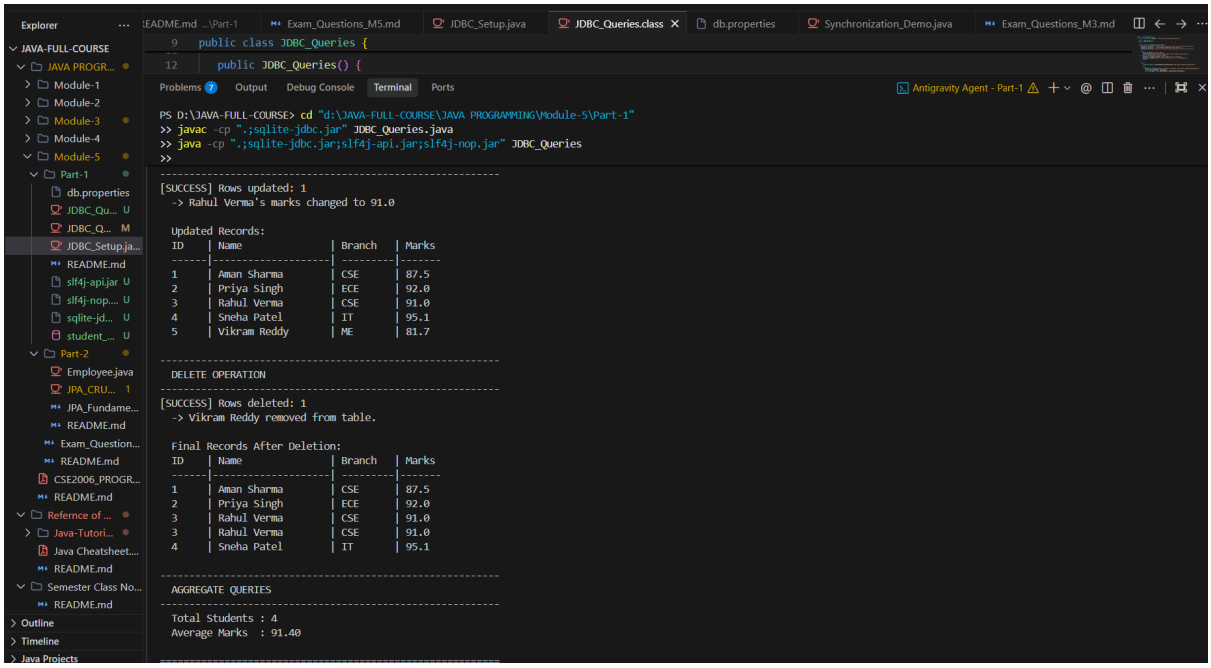


Figure 7: Output Part 2 – UPDATE, DELETE results, and aggregate queries

3 Conclusion

The program successfully demonstrates:

1. **JDBC Driver Loading** – Using `Class.forName()` to load the SQLite JDBC driver.
2. **Database Connection** – Establishing connection via `DriverManager.getConnection()`.
3. **CREATE TABLE** – Creating the `Students` table with appropriate schema.
4. **INSERT (Create)** – Adding 5 student records into the database.
5. **SELECT (Read)** – Querying all records and filtering with `WHERE` clause using `ResultSet`.
6. **UPDATE** – Modifying existing records (`Rahul Verma's marks updated to 91.0`).
7. **DELETE** – Removing a record (`Vikram Reddy` removed from the table).
8. **Aggregate Queries** – Computing `COUNT(*)` and `AVG(marks)`.
9. **Resource Management** – Using `try-with-resources` for automatic connection cleanup.

Submitted by: Mausam Kar — **Reg. No:** 24BAI10284